MATH 174 - Numerical Analysis

Dani Nguyen

Fall 2025

Contents

1 Error

1.1 Approximation

Let x be an exact number.
Let y be an approximation of x.

Def: The absolute error is $|x - y|$.
Def: The relative error is $\dfrac{|x-y|}{|x|}$,

1.2 Round-off Error

Usually a machine uses a finite amount of memory to store a number.
machine $\leftarrow \pi$
Suppose a machine can store some number of digits.

1. Allocate a digit for sign, positive or negative.
2. Allocate a number of digits for the mantissa.
3. Allocate a digit for sign of the exponent.
4. Allocate a number of digits for the exponent.

Under rounding in the case of a 4 digit mantissa, $\pi = 0.3142 \cdot 10^1$, this is the machine # representation of pi.

Def: Round-off error are all errors arising from the error between actual # and machine #.

Notation:

$$x = \text{real } \#$$
$$fl(x) = \text{machine } \# \text{ representation of } x$$

In an s-digit rounding machine,

Absolute error $= |x = fl(x)|$

Relative error $= \dfrac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \cdot 10^{1-5} = 5 \cdot 10^{-5}$

1.3 Floating Point Arithmetic

What happens under arithmetic operations? (+, -, ·, /)

Two things to watch out for:

1. Many operations can degrade error.
2. Catastrophic cancellation.

1.3.1 Degrading Error

number	machine
x	$fl(x)$
y	$fl(y)$
$x+y$	$fl(fl(x)+fl(y))$

The more nested $fl()$ there are, the larger the error will be. The same follows for -, ·, /.

When performing arithmetic, it temporarily allows for more digits than the mantissa allows.

Algorithmic "solutions"

$((x+y)+z)+w$

Worst case, 4 times the error.

$(x+y)+(z+w)$

Worst case, 3 times the error.

1.3.2 Catastrophic Cancellation

Ex: 3-digit rounding machine. Given 2 numbers: 0.312 and 0.311.

$$0.312 - 0.311 = 0.001 = 0.1 \cdot 10^{-2}$$
$$\text{Loss of significant digits!}$$

Ex: 5-digit rounding machine. Given 2 numbers:

$$\begin{aligned}
x &= 0.41626324 \\
y &= 0.41626323 \\
x - y &= 0.00000001 \\
\text{In machine:} & \\
fl(x) &= 0.41626 \\
fl(y) &= 0.41626 \\
fl(x) - fl(y) &= 0.00000 = 0 \\
fl(fl(x) - fl(y)) &= 0 \\
\text{Relative error:} & \\
\frac{|x - y - fl(fl(x) - fl(y))|}{|x - y|} &= \frac{|x - y - 0|}{|x - y|} \\
&= 1 = 100\% \text{ error!}
\end{aligned}$$

2 Speed

2.1 Dot Product

Consider dot product of vectors

$$\vec{x} \in \mathbb{R}^n, \vec{y} \in \mathbb{R}^n$$
$$\vec{x} \cdot \vec{y} = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$
$$= x_1 y_1 + x_2 y_2 + \ldots + x_n y_n$$

Count time-consuming operations:

1. Add/subtract
2. Multiply/divide
3. Comparisons
4. Memory access/storage

These are called FLOPS (floating point operations).

For the dot product, there are n multiplications and n-1 additions.
Adding these together, we get $2n - 1$ FLOPS.
Note that this expression is linear in n.

For large n, 2n dominates, $= \mathcal{O}(n) \approx$ constant times n.
Ex: For n vs $2n$, $2n$ will take twice as long.
Ex: For $\mathcal{O}(n^2)$, n vs $2n$, $2n$ will take four times as long.

2.2 Memory

Consider a matrix $A_{n\times n}$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} vs \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The square matrix uses n^2 memory locations, while the vector uses n memory locations. Each number also uses 8 bytes, creating another goal for algorithm development, reduce memory usage.

3 Gaussian Elimination

3.1 Triangular Matrices

Write a MATLAB function that when given $A\vec{x} = \vec{b}$, computes $\vec{x}$

$$A\vec{x} = \vec{b}$$

A is triangular and invertible

Let's say A is lower triangular. $a_{ij} = 0, \forall j > i$

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & 0 & \dots & 0 \\ a_{31} & a_{32} & a_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

Similarly, if A is upper triangular, $a_{ij} = 0, \forall j < i$.

Ex: $A\vec{x} = \vec{b}$, A is upper triangular.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

This gives us the following system of equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{33}x_3 &= b_3 \end{aligned}$$

So $x_3 = \frac{b_3}{a_{33}}, a_{33} \neq 0$ if A is invertible.

Then, $x_2 = \frac{b_2 - a_{23}x_3}{a_{22}}, a_{22} \neq 0$ if A is invertible.

Finally, $x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, a_{11} \neq 0$ if A is invertible.

This is called back substitution.
For $i = n, n-1, \dots, 1$

$$\begin{aligned} x_i &= \frac{b_i - (a_{i,i+1}x_{i+1} + \dots + a_{i,n}x_n)}{a_{ii}} \\ &= \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}} \end{aligned}$$

Ex: $A\vec{x} = \vec{b}$, A is lower triangular.

$$\begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

This gives us the following system of equations:

$$\begin{aligned} a_{11}x_1 &= b_1 \\ a_{21}x_1 + a_{22}x_2 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$

This is called forward substitution.
For $i = 1, 2, \ldots, n$

$$\begin{aligned} x_i &= \frac{b_i - (a_{i,1}x_1 + \ldots + a_{i,i-1}x_{i-1})}{a_{ii}} \\ &= \frac{b_i - \displaystyle\sum_{j=1}^{i-1} a_{ij}x_j}{a_{ii}} \end{aligned}$$

Now given a regular matrix, make it triangular without changing the solutions. Recall Gaussian elimination.

1. E_i (equation i) replaced by cE_i. Row scaling.
2. E_i swapped with E_j. Row swapping.
3. E_i replaced by $E_i + cE_j$. Row addition.

3.1.1 Speed of Back Substitution

For $i = n, n-1, \ldots, 1$

$$x_i = \frac{b_i - \displaystyle\sum_{j=i+1}^{n} u_{ij}x_j}{u_{ii}}$$

$$\begin{aligned} i = n &\qquad 1 \text{ flop} \\ n-1 &\qquad 3 \text{ flops} \\ \vdots &\qquad \vdots \\ 1 &\qquad 2n-1 \text{ flops} \end{aligned}$$

$$\sum_{j=1}^{n}(2j-1) = 2\sum_{j=1}^{n}(j) - \sum_{j-1}^{n} 1 = 2\left(\frac{n(n+1)}{2}\right) - n = n^2$$

With a large n, the term n^2 dominates, meaning $\mathcal{O}(n^2)$

3.2 Augmented Matrices

Gaussian Elimination on same matrices and different RHS comes up often in applications and algorithms.

3.2.1 LU Factorization

$$A = LU$$

where L is lower triangular with 1's in the diagonal and U is upper triangular

The important entries of L are called multipliers. These entries can be found with the negative of the coefficients of the steps of Gaussian elimination.

$$\begin{aligned} L\vec{y} &= \vec{b} \qquad && \text{get y from forward substitution} \\ U\vec{x} &= \vec{y} \qquad && \text{get x from back substitution} \end{aligned}$$

3.3 Matrix Inverses

Matrix-vector multiplication:

$$(A\vec{x})_i = \sum_{j=1}^{n} a_{ij}x_j$$

which gives $2(n-1)n$ flops total

Backwards substitution uses n^2 flops.
Forward substitution uses $n^2 - n$ flops. Because we're working with L, which has 1's on the diagonal, we save n divisions.

In summary, getting A^{-1} to solve $A\vec{x} = \vec{b}$ is slower. Also, A^{-1} has memory storage issues for certain matrices. For example, a tridiagonal matrix, even though it has zeroes on most of its entries, A^{-1} has no zeroes.

3.3.1 Gaussian Elimination FLOP count

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

This step has 1 division, 1 multiplication, 1 addition/subtraction.
In total there are $2n-1$ total flops.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2n} \\ 0 & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

Repeat this for $n-1$ rows (minus 1 because we are excluding the first row).
We can then repeat this for the inner block matrix.

$$(2n-1)(n-1) + (2(n-1)-1)((n-1)-1) + (2(n-2)-1)((n-2)-1)\dots$$

$$\sum_{j-1}^{n} j = \frac{n(n+1)}{2}$$

$$\sum_{j-1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6}$$

4 Gaussian Elimination With Pivoting

But what about $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ Gaussian elimination fails on first step. Can we use another elementary row operation, namely, row swapping?

In general,

$$\begin{bmatrix} 0 & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

4.1 Error

Find a nonzero element a_{i1} then perform $E_1 \leftrightarrow E_2$. We have to do this before each elimination step. However, there is error involved.

$$\left[\begin{array}{cc|c} \epsilon & 1 & 1 \\ \times & \times & \times \end{array}\right] \to \tilde{x}_2 = x_2 + \delta$$
$$\to \tilde{x}_1 = \frac{1 - \tilde{x}_2}{\epsilon} = \frac{1 - x_2 - \delta}{\epsilon} = \frac{1 - x_2}{\epsilon} - \frac{\delta}{\epsilon}$$

Note that in this situation, choosing an x_1 that is small leads to a small ϵ which leads to a large error. We have to be careful when swapping rows.

$$\left[\begin{array}{cc|c} \epsilon & 1 & 1 \\ 0 & \times & \times \end{array}\right]$$

$$\begin{aligned} \tilde{x}_2 &= x_2 + \delta \\ |\delta| &= |\tilde{x}_2 - x_2| \\ \tilde{x}_1 &= x_1 - \frac{\delta}{\epsilon} \\ \left|\frac{\delta}{\epsilon}\right| &= |\tilde{x}_1 - x_1| \end{aligned}$$

4.1.1 Section 7.5

$$\begin{aligned} A\vec{x} &= \vec{b} \\ A(\vec{x} + \vec{\delta x}) &= \vec{b} + \delta\vec{b} \\ A\vec{\delta x} &= \delta\vec{b} \\ \vec{\delta x} &= A^{-1}\delta\vec{b} \end{aligned}$$

We can use a norm to measure the size of error vectors.

$$\begin{aligned} ||\vec{\delta x}|| = ||A^{-1}\delta\vec{b}|| &\leq ||A^{-1}||\ ||\delta\vec{b}|| \\ \frac{||\vec{\delta x}||}{||\vec{x}||} &\leq ||A||\ ||A^{-1}|| \frac{||\delta\vec{b}||}{||\vec{b}||} \end{aligned}$$

$$\text{relative error} \leq \text{condition } \# \times \text{relative size of change of } \vec{b}$$

Ex: Suppose you need to solve accurately $A\vec{x} = \vec{b}$ and someone gives you the approx $\tilde{x}$, is it accurate?

We can use the residual vector.

$$\begin{aligned} \vec{r} &= \vec{b} - A\tilde{x} \\ A\tilde{x} &= \vec{b} - \vec{r} \end{aligned}$$

If A is well-conditioned (small condition number), then $\frac{||\vec{r}||}{||\vec{b}||}$ is small and $\frac{||\vec{x}-\tilde{x}||}{||\vec{x}||}$ small.

If A is ill-conditioned (large condition number), then we don't know.

4.2 Row Swapping

$$PA = LU$$

where PA is A with rows swapped. P is the identity matrix with its rows swapped to denote the row swapping operations done on A.

Now solving LU factorization:

$$
\begin{aligned}
A\vec{x} &= \vec{b} \\
PA\vec{x} &= P\vec{b} \\
LU\vec{x} &= P\vec{b} \\
L\vec{y} &= P\vec{b} \quad \text{forward sub} \\
U\vec{x} &= \vec{y} \qquad \text{back sub}
\end{aligned}
$$

4.3 Speed

In an $n \times n$ matrix, it takes $n-1$ comparisons for the first step, $n-2$ for the second step, and so on.

$$
\begin{aligned}
\sum_{j=1}^{n-1} j &= \frac{(n-1)n}{2} \\
&= \mathcal{O}(n^2)
\end{aligned}
$$

This is negligible additional amount of work for large n compared to $\mathcal{O}(n^3)$ of Gaussian elimination.

Even safer than row partial pivoting in terms of error:

1. Scaled row partial pivoting:

2. Complete pivoting: row swaps and column swaps to move the largest element to pivot element.

5 Approximations

5.1 Fixed Point Problems

Solve an equation $g(x) = h(x)$
Modify to $f(x) = x$

Now we have the fixed point problem, which we can try to solve using fixed point iterations.

Start with guess $x^{(0)}$
Then iterate $x^{(k+1)} = f(x^{(k)})$
The $x^{(k)}$ are approximations of the fixed point.

We can also write $A\vec{x} = \vec{b}$ as fixed point problem using splitting.

$$\begin{aligned} A &= M - N \\ (M - N)\vec{x} &= \vec{b} \\ M\vec{x} &= N\vec{x} + \vec{b} \\ \vec{x} &= M^{-1}(N\vec{x} + \vec{b}) \end{aligned}$$

where the RHS is $f(\vec{x})$

Choices for M, N:

Let $A = D - L - U$, where D is diagonal, L is lower triangular, U is upper triangular. We have

1. Jacobi iterative method.
2. Gauss-Seidel iterative method.

5.1.1 Jacobi Iterative Method

$$\begin{aligned} M &= D \\ N &= L + U \\ D\vec{x} &= (L + U)\vec{x} + \vec{b} \end{aligned}$$

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)}}{a_{ii}}$$

5.1.2 Gauss-Seidel Iterative Method

$$\begin{aligned} M &= D - L \\ N &= U \\ (D - L)\vec{x} &= U\vec{x} + \vec{b} \end{aligned}$$

$$x_i^{(k+1)} = \frac{b_i - \sum_{j>i} a_{ij}x_j^{(k)} - \sum_{j<i} a_{ij}x_j^{(k+1)}}{a_{ii}}$$

5.2 Convergence

Jacobi & Gauss-Seidel are iterative methods. They start with a guess, which improves their approximations at each step. We want $\vec{x}^{(k)}$ sequence of approximations to converge to the exact solution.

Practically, we need a stopping condition. Here are our choices:

1. Stop after certain number of steps
2. Stop when residual is small, where the residual is $\vec{r} = \vec{b} - A\vec{x}$, $||\vec{r}|| \leq$ tolerance.
3. Stop when the algorithm slows down, $||x^{(k+1)} - x^{(k)}|| \leq$ tolerance.

$$\begin{aligned}
&A = M - N \\
\text{Jacobi} \quad & M = D \\
\text{Gauss-Seidel} \quad & M = D - L \\
A\vec{x} = \vec{b} &\rightarrow (M - N)\vec{x} = \vec{b} \\
&\rightarrow M\vec{x} = N\vec{x} + \vec{b} \\
&\rightarrow \vec{x} = M^{-1}(N\vec{x} + \vec{b}) \\
&\rightarrow \vec{x}^{(k+1)} = M^{-1}(N\vec{x}^{(k)} + \vec{b}) \\
&\rightarrow \vec{x}^{(k+1)} = \mathsf{T}\vec{x}^{(k)} + \vec{c} \\
&\text{where } \mathsf{T} \text{ is the iteration matrix, } M^{-1}N \\
&\text{and } \vec{c} \text{ is } M^{-1}\vec{b}
\end{aligned}$$

An iterative method using $\vec{x}^{(k+1)} = \mathsf{T}\vec{x}^{(k)} + \vec{c}$ converges if $\varphi(\mathsf{T}) < 1$, where $\varphi(\mathsf{T})$ is the spectral radius of T.

Def: Spectral radius $\varphi(A) = \max\{|\lambda_1|, |\lambda_2|, \ldots, |\lambda_n|\}$

$$||\vec{e}^{\,k+1}|| \approx \varphi(T)||\vec{e}^{\,k}$$
$$||\vec{e}^{\,k}|| \approx \varphi(T)^k||\vec{e}^{\,(0)}||$$

Let A $= \begin{bmatrix} a & b \\ b & c \end{bmatrix}$

$T_J = \begin{bmatrix} 0 & \frac{-b}{a} \\ \frac{-b}{c} & 0 \end{bmatrix}$

$T_{GS} = \begin{bmatrix} 0 & \frac{-b}{a} \\ 0 & \frac{b^2}{ac} \end{bmatrix}$

$\det T_J = \begin{bmatrix} -\lambda & \frac{-b}{a} \\ \frac{-b}{c} & -\lambda \end{bmatrix} = \lambda^2 - \frac{b^2}{ac}$

$\det T_{GS} = \begin{bmatrix} -\lambda & \frac{-b}{a} \\ 0 & \frac{b^2}{ac} - \lambda \end{bmatrix} = \lambda(\lambda - \frac{b^2}{ac})$

$\lambda_J = \pm\sqrt{\frac{b^2}{ac}} = \varphi(T_J)$

$\lambda_{GS} = 0$ or $\frac{b^2}{ac}$, $\varphi(T_{GS}) = \frac{b^2}{ac}$

If they converge, then $\frac{b^2}{|ac|} < 1$, and when this condition is true, Gauss-Seidel's method has a lower spectral radius than Jacobi, since $x < \sqrt{x}$ for x < 1.

However, there are examples for general matrices where Jacobi's method converges and Gauss-Seidel's method does not. So for general matrices, we do not know which one is faster. In practice, however, Gauss-Seidel is almost always faster.

Other iterative methods:

1. Richardson's
2. Successive Over-Relaxation (SOR) - Gauss-Seidel is in this class
3. Conjugate Gradient (not splitting)
4. GMRES

5.2.1 Richardson's Iterative Method

$$
\begin{aligned}
A\vec{x} &= \vec{b} \\
\omega A\vec{x} &= \omega\vec{b} \\
\omega A &= I - I + \omega A \\
&= I - (I - \omega A) \text{ where I is M, and } (I - \omega A) \text{ is N}
\end{aligned}
$$

$$
\begin{aligned}
x^{k+1} &= M^{-1}(Nx^k + \omega b) \\
&= Nx^k + \omega b \\
&= (I - \omega A)x^k + \omega b \\
x_i^{k+1} &= x_i^k - \omega \sum_{j=1}^{n} a_{ij}x_j^k + \omega b_i
\end{aligned}
$$

6 Eigenvalues

6.1 Finding Eigenvalues of a Matrix

Given $A \in \mathbb{R}^{n\times n}$, we want to find either an eigenvalue or all eigenvalues.

Suppose we have $\lambda_1, \lambda_2, \ldots, \lambda_n$, where $|\lambda_1| \geq |\lambda_2| \geq \ldots \geq |\lambda_n|$

Consider the special case where A has a linearly independent set of eigenvectors (which is true when A is symmetric).

Let our eigenvectors be $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_n$ with $\vec{x}_i$ corresponding to λ_i.

So $A\vec{x}_i = \lambda_i\vec{x}_i$, with the iterative method, we can start with initial guess $\vec{x}^{\,(0)}$

$$
\begin{aligned}
\vec{x}^{\,(0)} &= \alpha_1\vec{x}_1 + \alpha_2\vec{x}_2 + \ldots + \alpha_n\vec{x}_n \\
A\vec{x}^{\,(0)} &= A\alpha_1\vec{x}_1 + A\alpha_2\vec{x}_2 + \ldots + A\alpha_n\vec{x}_n \\
A\vec{x}^{\,(0)} &= \alpha_1 A\vec{x}_1 + \alpha_2 A\vec{x}_2 + \ldots + \alpha_n A\vec{x}_n \\
A\vec{x}^{\,(0)} &= \alpha_1\lambda_1\vec{x}_1 + \alpha_2\lambda_2\vec{x}_2 + \ldots + \alpha_n\lambda_n\vec{x}_n \\
A^k\vec{x}^{\,(0)} &= \alpha_1\lambda_1^k\vec{x}_1 + \alpha_2\lambda_2^k\vec{x}_2 + \ldots + \alpha_n\lambda_n^k\vec{x}_n \\
&\approx \alpha_1\lambda_1^k\vec{x}_1 \text{ which is our approximate eigenvector}
\end{aligned}
$$

The convergence speed of this approximation is related to $\left|\frac{\lambda_2}{\lambda_1}\right|^k$ (compared to $\varphi(\mathsf{T}^k)$).

There are two main issues with this approximation.

1. $\lambda_1^k \alpha_1 \vec{x}_1 \to \infty$ or 0 which is not our eigenvector.
2. $\lambda_1^k \alpha_1 \vec{x}_1 \to 0$ if $\alpha_1 = 0$

6.2 Power Method

1. Start with initial guess $\vec{x}^{(0)}$.
2. Find component largest in magnitude.
3. Replace $\vec{x}^{(0)}$ by $\vec{x}^{(0)}$ divided by largest component.
4. Perform $A\vec{x}^{(0)}$ and call result $\vec{x}^{(1)}$
5. Normalize $\vec{x}^{(1)}$, this is our approximate eigenvector.
6. Repeat from step 4.

But what about other eigenvalues?

6.2.1 Inverse Power Method

Choose a q which will be our initial guess for eigenvalue.
A^{-1} has eigenvalues $\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \ldots, \frac{1}{\lambda_n}$
with $|\frac{1}{\lambda_n}| \geq |\frac{1}{\lambda_{n-1}}| \geq \ldots \geq |\frac{1}{\lambda_1}|$
Power method on A^{-1} gives $\frac{1}{\lambda_n}$